Networks & Distributed Programming

The Transport Layer

Lecture 14



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II



DIETI

UNINA

Transport Layer

What happens beneath the socket

We have seen how applications use sockets:

- bind() associates a socket with a local address and port.
- connect() identifies a remote endpoint.
- accept() returns a new socket for each incoming TCP connection.

But how does an incoming segment/UDP datagram find the right socket?

This brings us to the **transport layer**:

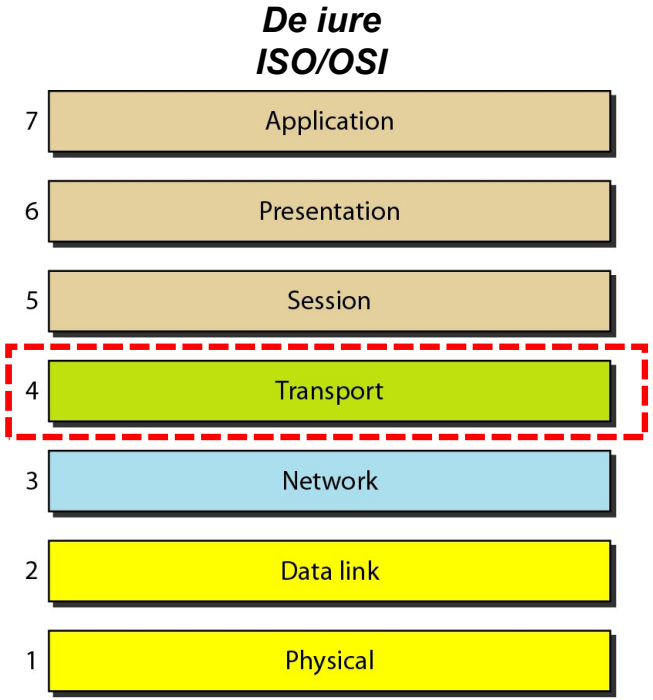
- Multiplexing and demultiplexing: delivering data to the right socket.
- UDP and TCP: two different transport services.
- Interaction with IP: how transport segments travel across the network.

Transport Layer

From Application to Transport

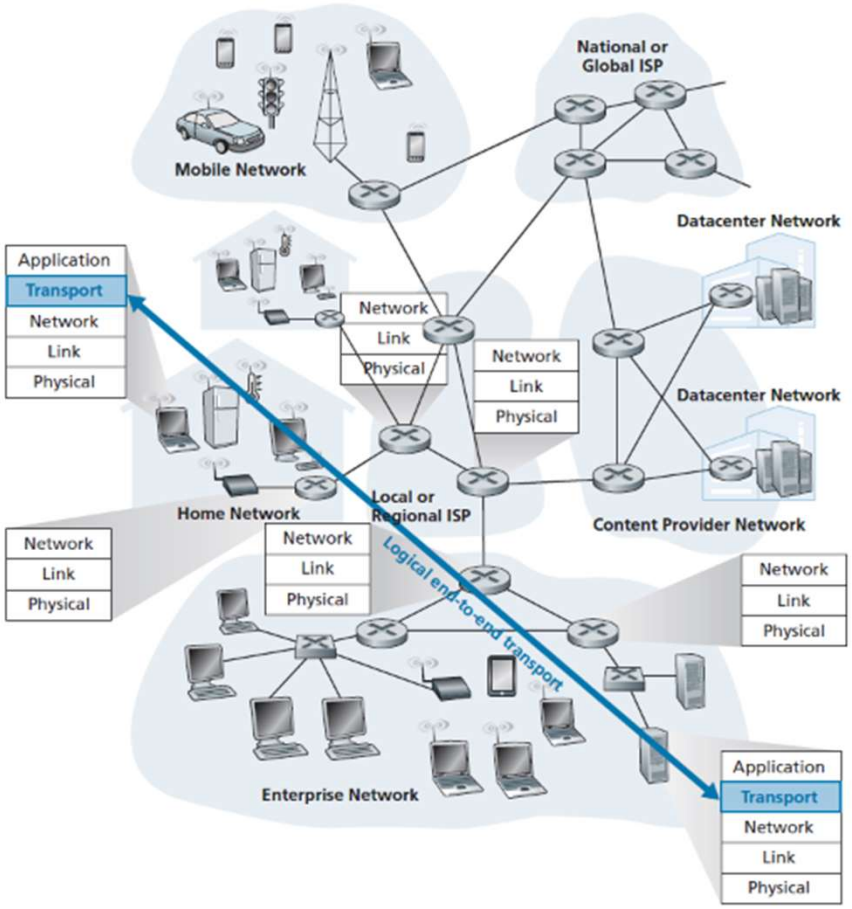
- A transport-layer protocol provide **logical communication between application processes** running on different hosts.
  - From the application's perspective, hosts running **the processes looks directly connected** as if they were on the same machine (even if they are on opposite sides of the planet).
- **At the sender:**
  - TCP organizes the application byte stream into **segments**; UDP encapsulates application messages into **datagrams**.
  - Each segment/datagram has a **transport header** and is passed to IP.
- **At the receiver:**
  - The transport layer **demultiplexes** incoming segments/datagrams to the appropriate socket.
  - TCP delivers an ordered byte stream; UDP delivers individual datagrams.

Note: TCP segmentation and IP fragmentation are different mechanisms operating at different layers.



Transport Layer

From Application to Transport



De iure ISO/OSI

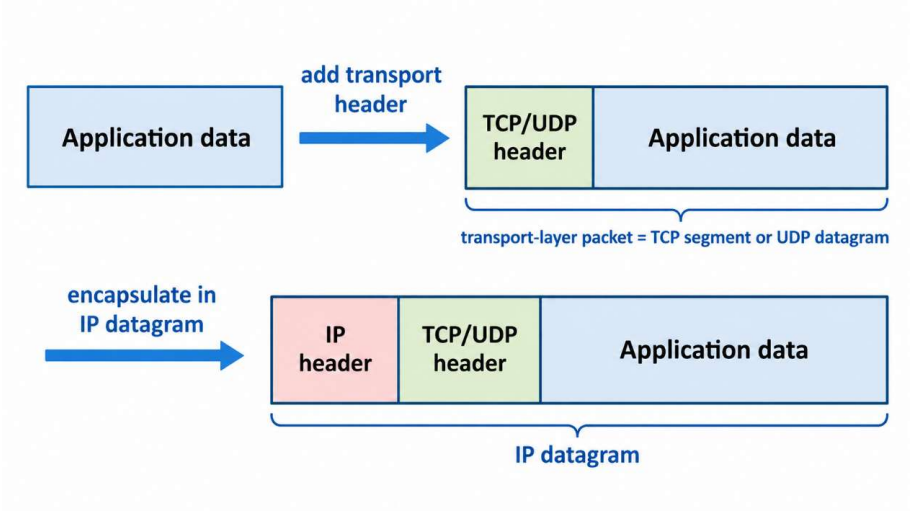
7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data link
1	Physical

Transport Layer

Logical communication

From the application's point of view the two hosts look directly connected, whatever sits between them: routers, links, etc.

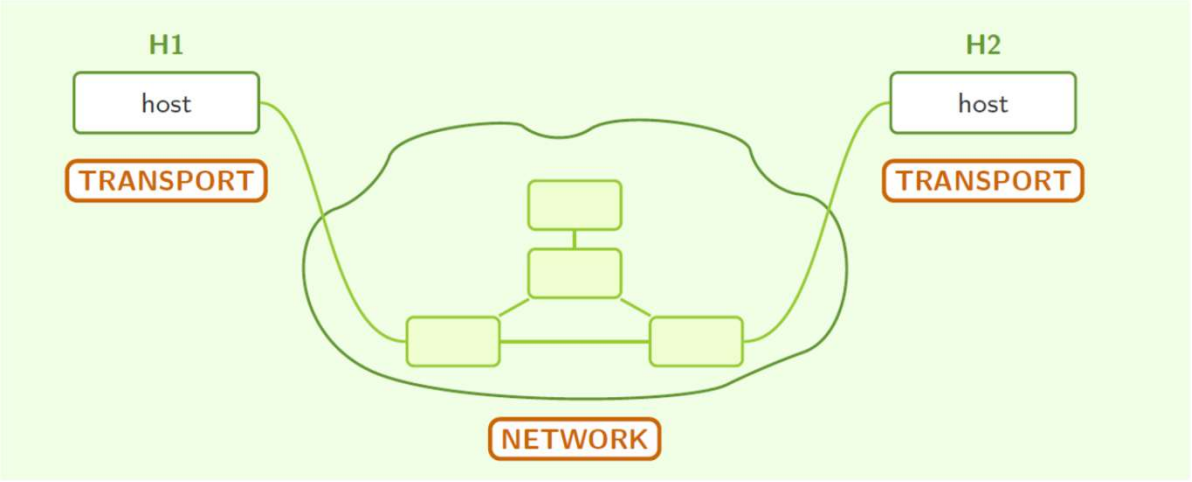
- The **sending transport layer** takes application data and adds a **transport-layer header**, creating a transport-layer packet: a TCP segment or a UDP datagram.
- TCP treats application data as a **byte stream** and divides it into **segments**; UDP preserves **application message boundaries**.
- Each transport-layer packet is passed to the network layer, which encapsulates it into an IP datagram and attempts best-effort delivery.



Transport Layer

Logical communication

From the application's point of view the two hosts look directly connected, whatever sits between them: routers, links, etc.



Transport protocols are implemented in the end systems.
Routing depends on the topology, so the network layer must run where the topology is — on the nodes.
Hence the two sentences that hold the whole layer together:
transport connects processes, the network connects hosts.

Transport Layer

Responsibilities of Transport Layer

- Transport-layer protocols (UDP and TCP) have four main responsibilities:
  1. **Process-to-process delivery:** messages are delivered independently of where these processes are.
     - **different from host-to-host delivery** (i.e., between two different machines), which is responsibility of the lower-level IP protocol.
  2. **Integrity checking:** by including error detection fields into the segments' headers.
  3. **Reliable data transfer**: ensuring that data is delivered from sending process to receiving process, correctly and in order.
  4. **Congestion/flow control**: it prevents connection from swamping network devices (links or routers) with an excessive amount of traffic. This service is useful to improve performance and **is very beneficial to the whole (internet) network**.

- UDP (faster but simpler) provides **only the first two services**, while TCP provides all the four ones.

Transport Layer

Process-to-process Delivery

- At the application level, **several processes can be accessing the network at the same time**.
  - E.g., listening to Spotify music, downloading files, navigating web sites, all together.

- A **process may also be connected to multiple processes** at the same time:
  - E.g., proxy servers, mail servers, etc., all may be connected to multiple remote processes.

- Sockets provide the endpoints for process-to-process communication:
  - The transport layer delivers incoming data to the appropriate socket, from which the application process can read it.
  - A process may use multiple sockets, and a server process may handle multiple concurrent connections.
  - Therefore, there is no one-to-one correspondence between processes and connections.

Transport Layer

Process-to-process Delivery

- Since multiple application sockets may communicate over the network at the same time, the transport layer must:
  - **Multiplex** outgoing data from different sockets onto the underlying network service.
  - **Demultiplex** incoming data to the appropriate receiving socket.

- **Multiplexing** (sender): collects data from different sockets, adds transport-layer headers (including port numbers), and passes the resulting segments or datagrams to the network layer.
- **Demultiplexing** (receiver): examines the information in received packets and delivers their data to the appropriate socket.

Port numbers play a key role in identifying the communicating endpoints.

Transport Layer

Ports

- Transport protocols use port numbers to distinguish comm. endpoints and support multiplexing/demultiplexing.
  - Source and destination ports are included in TCP and UDP headers.
  - A port number is a 16-bit integer, ranging from 0 to 65535.
- Port numbers are classified by **IANA** (Internet Assigned Numbers Authority), available at http://www.iana.org.
  - Well-known ports: 0–1023.
  - Registered ports: 1024–49151. (e.g., MySQL 3306/TCP)
  - Dynamic/private ports: 49152–65535.
- When we develop a new network application, we must assign port numbers to sockets accordingly.

Port	Usage
20	File Transfer Protocol (FTP) Data Transfer
21	File Transfer Protocol (FTP) Command Control
22	Secure Shell (SSH)
23	Telnet - Remote login service, unencrypted text messages
25	Simple Mail Transfer Protocol (SMTP) E-mail Routing
53	Domain Name System (DNS) service
80	Hypertext Transfer Protocol (HTTP) used in World Wide Web
110	Post Office Protocol (POP3) used by e-mail clients to retrieve e-mail from a server
119	Network News Transfer Protocol (NNTP)
123	Network Time Protocol (NTP)
143	Internet Mail Access Protocol (IMAP) Management of Digital Mail
161	Simple Network Management Protocol (SNMP)
443	HTTP Secure (HTTPS) HTTP over TLS/SSL

Transport Layer

Ports: Nmap

- To **check port usage** (and more) on a Linux machines we can use the nmap command.
- Nmap (Network Mapper) is a **network scanning utility** basically created to map (discover) hosts and services on a network.
- It can be used to probe ports of hosts **in order to detect open ones** (on which an application is listening) and possibly the associated services.

- Usage:
  - Port-scan of a given target:
    - $ sudo nmap [target address]
  - Probe port to determine services:
    - $ sudo nmap –sV [target address]
  - Check first N top used ports:
    - $ sudo nmap –top-port [N] [target address]

```
hargalaten@hargalaten-Lenovo-B50-80:~$ nmap --top-port 10 www.google.com
Starting Nmap 7.80 ( https://nmap.org ) at 2023-10-13 15:27 CEST
Nmap scan report for www.google.com (142.250.180.132)
Host is up (0.021s latency).
Other addresses for www.google.com (not scanned): 2a00:1450:4002:809::2004
rDNS record for 142.250.180.132: mil04s43-in-f4.1e100.net

PORT     STATE    SERVICE
21/tcp   filtered ftp
22/tcp   filtered ssh
23/tcp   filtered telnet
25/tcp   filtered smtp
80/tcp   open     http
110/tcp  filtered pop3
139/tcp  filtered netbios-ssn
443/tcp  open     https
445/tcp  filtered microsoft-ds
3389/tcp filtered ms-wbt-server

Nmap done: 1 IP address (1 host up) scanned in 1.33 seconds
hargalaten@hargalaten-Lenovo-B50-80:~$
```

Note: state can be open or closed, filtered or unfiltered (which means unable to be scanned).

Transport Layer

Ports: Nmap

Nmap commands:

- nmap scanme.nmap.org
  Scan the 1000 most common TCP ports.
- nmap --top-ports 10 scanme.nmap.org
  Scan the 10 most common TCP ports.
- nmap -p 22,80,443 scanme.nmap.org
  Scan only the specified ports (-p).
- nmap -sV --version-light -p 22,80 scanme.nmap.org
  Attempt to identify services and software versions (service version detection).

Example output (illustrative):

```
PORT     STATE    SERVICE   VERSION
22/tcp   open     ssh       OpenSSH
25/tcp   filtered smtp
80/tcp   open     http      Apache httpd
443/tcp  closed   https
```

Output fields:

- PORT: port number and transport protocol.
- STATE: open, closed, or filtered.
- SERVICE: associated or detected application service.
- VERSION: detected software and possibly its version.

- Note: Without -sV, service names are generally inferred from conventional port assignments. With -sV, Nmap probes open ports to identify the actual service.

Transport Layer

Ports: Nmap

```
$ nmap --top-ports 6 scanme.nmap.org
Nmap scan report for scanme.nmap.org (45.33.32.156)
PORT     STATE    SERVICE
21/tcp   filtered ftp
22/tcp   open     ssh
23/tcp   filtered telnet
25/tcp   filtered smtp
80/tcp   open     http
110/tcp  closed   pop3
```

- `open` means something is bound and listening; `closed` means the host answered, and nothing is there.
- `filtered` means no answer came back — a firewall swallowed the probe, so the state is unknown.
- `nmap -sV` goes further and asks the port what it is, which is also the first move of whoever is looking for a way in.

Transport Layer

Ports: nmap vs ss

Nmap — Network perspective

$ nmap --top-ports 10 scanme.nmap.org

- Scans ports on a target host by sending network probes.
- Identifies ports as open, closed, or filtered.
- With -sV, attempts to identify application services.

ss — Local operating system perspective

$ sudo ss -tulnp

- Displays local TCP and UDP sockets.
- Shows local addresses, port numbers, and socket states.
- Can identify the processes associated with sockets.

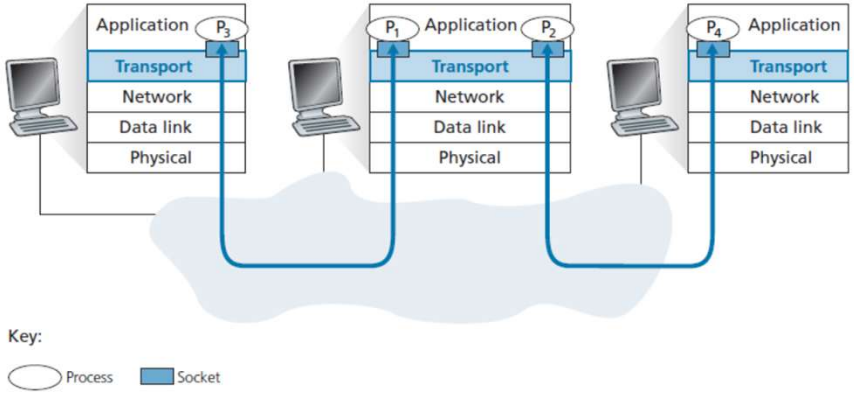
Options:

- -t TCP sockets; -u UDP sockets
- -l listening sockets (and applicable UDP sockets)
- -n numeric addresses and ports
- -p associated processes

Key difference: Nmap discovers network-accessible services, while ss inspects sockets directly through the operating system kernel.

Transport Layer

Multiplexing and Demultiplexing



Multiplexing:

- Multiple sockets on Host A may send data simultaneously.
- The transport layer collects data from these sockets and encapsulates them into separate segments, including source and destination port numbers.

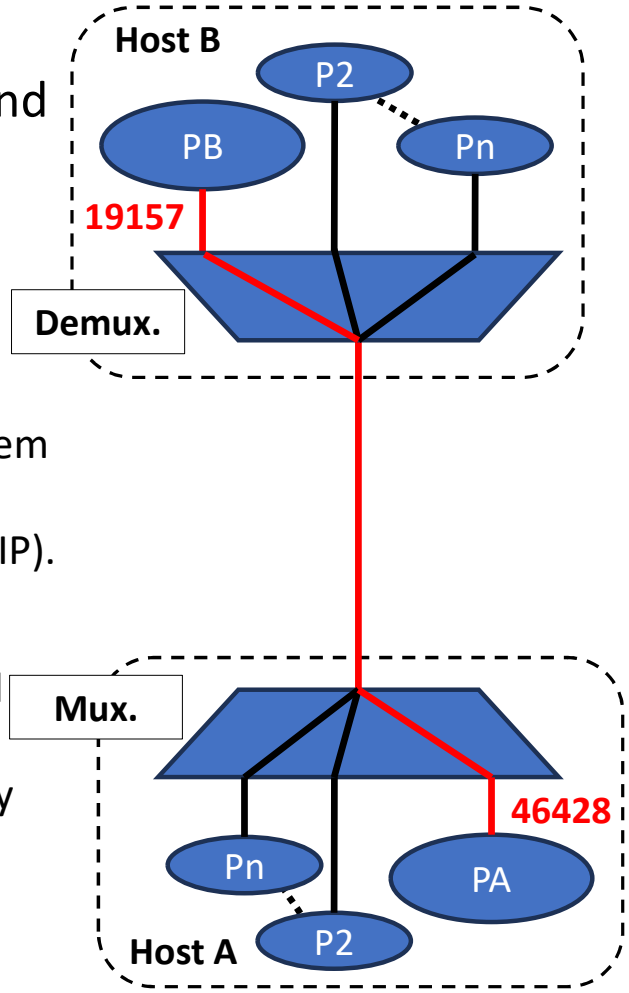
Demultiplexing:

- The receiving transport layer examines the destination port number (and the local IP address, when relevant).
- The transport layer then passes the segment to the appropriate socket by examining the segment's destination port number.

Transport Layer

Multiplexing and Demultiplexing

Assume we have a process in Host A (port 46428) that wants to send a message to a process (port 19157) in Host B.



- **Multiplexing:**
  - Multiple sockets on Host A may send data simultaneously.
  - The transport layer collects data from these sockets and encapsulates them into separate segments, including source and destination port numbers.
  - These packets are passed to the same underlying network-layer service (IP).
- **Demultiplexing:**
  - The receiving transport layer examines the destination port number (and the local IP address, when relevant).
  - The transport layer then passes the segment to the appropriate socket by examining the segment's destination port number.

Transport Layer

Multiplexing/Demultiplexing in UDP

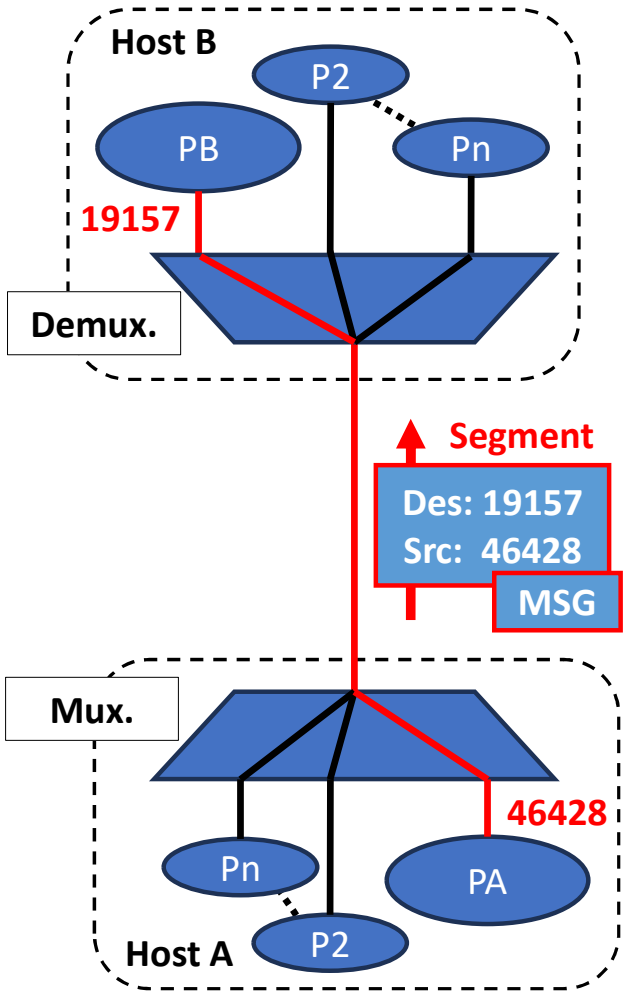
- In a **C UDP client**, socket is created as follows:

```
sockfd = socket(AF_INET, SOCK_DGRAM, 0);
```

- The socket is typically not bound to a specific source port, it is the OS that selects a free one (e.g., 46428) but we can always bind it to a specific port.

- Destination port/address is set by passing a suitable sockaddr_in structure to the sendto function:

```
servaddr.sin_port = htons(19157);
servaddr.sin_addr.s_addr = inet_addr( address_of_B );
...
sendto(sockfd, (const char *)msg, strlen(msg), 0,
        (const struct sockaddr *) &destaddr,
        sizeof(destaddr);
```



Transport Layer

Multiplexing/Demultiplexing in UDP

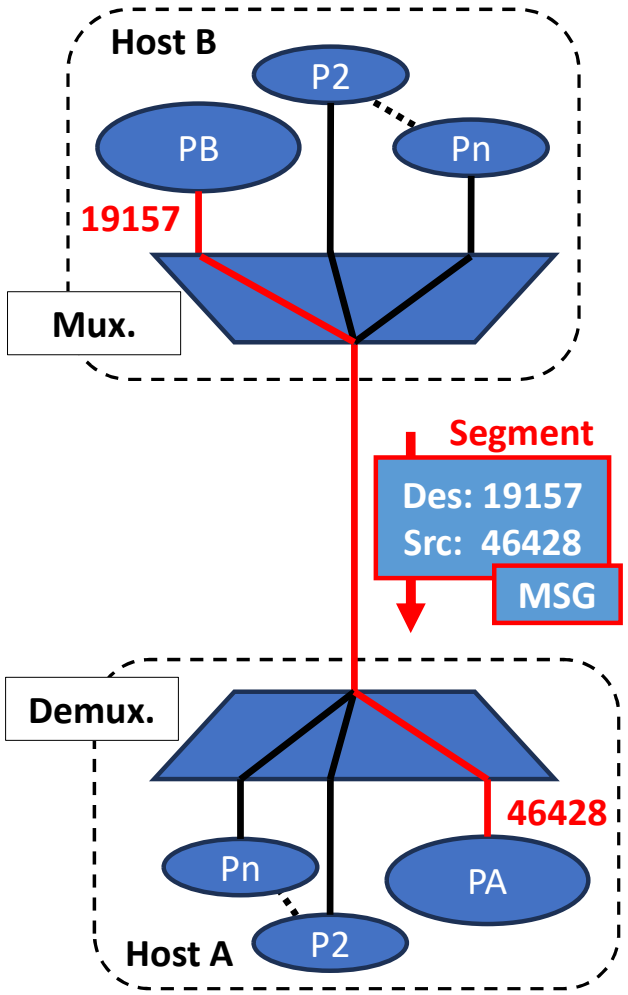
- In a **C UDP server**, socket is created and bound as follows:

```
sockfd = socket(AF_INET, SOCK_DGRAM, 0);
…
servaddr.sin_addr.s_addr = INADDR_ANY;
servaddr.sin_port = htons(19157);
…
bind(sockfd, (const struct sockaddr *)&servaddr, sizeof(servaddr));
```

- **Destination port/address is retrieved through the sockaddr_in structure** from the recvfrom function and used in the reply message:

```
recvfrom(sockfd, (char *)msg, 1024, 0,
        ( struct sockaddr *) &cliaddr, &len);
…
sendto(sockfd, (const char *)msg, strlen(msg), 0,
        (const struct sockaddr *) &cliaddr, sizeof(cliaddr);
```

- The reply will be sent back **whatever the client is**.



Transport Layer

Multiplexing/Demultiplexing in UDP

- In the usual connectionless UDP demultiplexing, incoming datagrams are delivered to the appropriate socket based on two fields:
  - Destination address (IP).
  - Destination port.

- We can use the same socket to send a return message **back to multiple source ports/addresses**.

- In the example, the cliaddr structure is filled by the recvfrom function, so we can use it as a destination address for the following sendto function, independently on who the host is.

Transport Layer

UDP: Destination Port Unreachable

What happens if a UDP datagram is sent to a port with no receiving socket?

- The receiving host's UDP layer attempts to demultiplex the datagram using its destination information.
- If no matching socket exists, the UDP datagram is discarded.
- The host normally generates an ICMP Destination Unreachable (Port Unreachable) message to notify the sender.

What happens at the sender?

- The operating system may report the error to the application (e.g., ECONNREFUSED on Linux).
- However, ICMP error messages are not guaranteed to be delivered.
- If no error or application response is received, a blocking recvfrom() may wait indefinitely unless a timeout is configured.

Key point: UDP does not guarantee delivery or provide a connection-establishment mechanism. Applications must handle missing responses and timeouts themselves.

Transport Layer

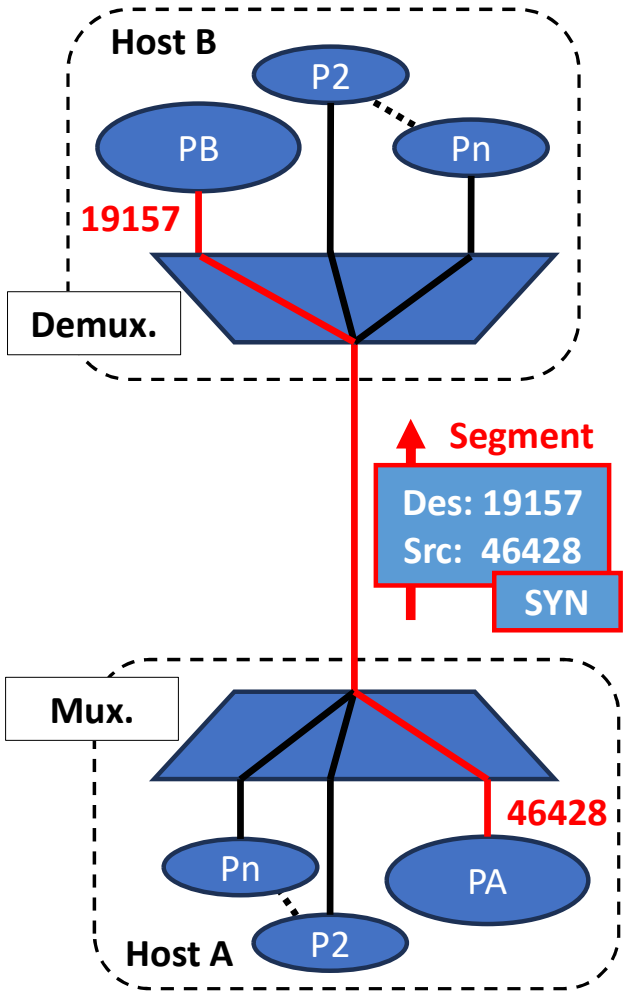
Multiplexing/Demultiplexing in TCP

- In TCP communication the server application **has a welcoming socket**, that waits for connection establishment requests from clients on a specific port number (19157 as before).

- The TCP client creates a socket and **sends a connection establishment request segment** to the host specified in the sockaddr_in structure (servaddr in this case):

```
sockfd = socket(AF_INET, SOCK_STREAM, 0);
…
servaddr.sin_port = htons(19157);
servaddr.sin_addr.s_addr = inet_addr( address_of_B );
…
connect(sockfd, (struct sockaddr*)&servaddr,
sizeof(servaddr));
```

- A **connection-establishment request is just a TCP segment** with a destination port number (19157 here) and a special connection-establishment bit set in the TCP header (SYN = 1).

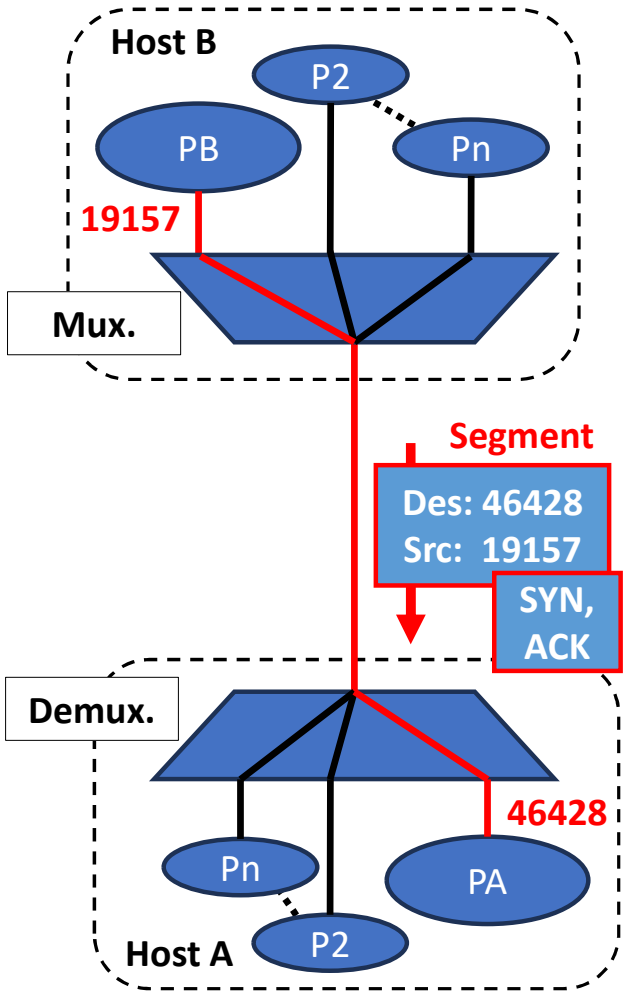


Transport Layer

Multiplexing/Demultiplexing in TCP

- When the Host B receives the incoming connection-request segment (with destination port 19157), its TCP layer uses the destination port to identify the appropriate listening socket.
- The connection is established through a **three-way handshake**:
  - SYN: The client sends a TCP segment with **SYN = 1** to request a connection.
  - SYN-ACK: The server replies with a segment having **SYN = 1 and ACK = 1**.
  - ACK: The client acknowledges the server's SYN by sending a segment with ACK = 1.
- After the handshake, the TCP connection is established and application data can be exchanged.

Note: The three segments carry the source and destination port numbers identifying the communicating endpoints.



Transport Layer

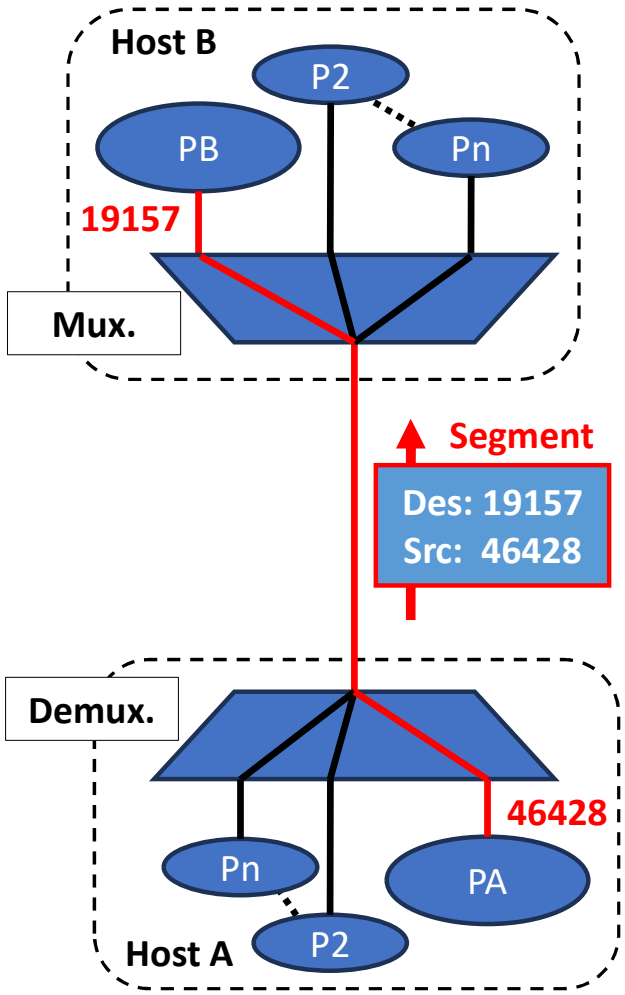
Multiplexing/Demultiplexing in TCP

- Once the three-way handshake is completed, the server application can obtain a new connected socket by calling accept() on the listening socket:

```
new_socket = accept(sockfd,
        (struct sockaddr*)&cliaddr, (socklen_t*)&addrlen)
```

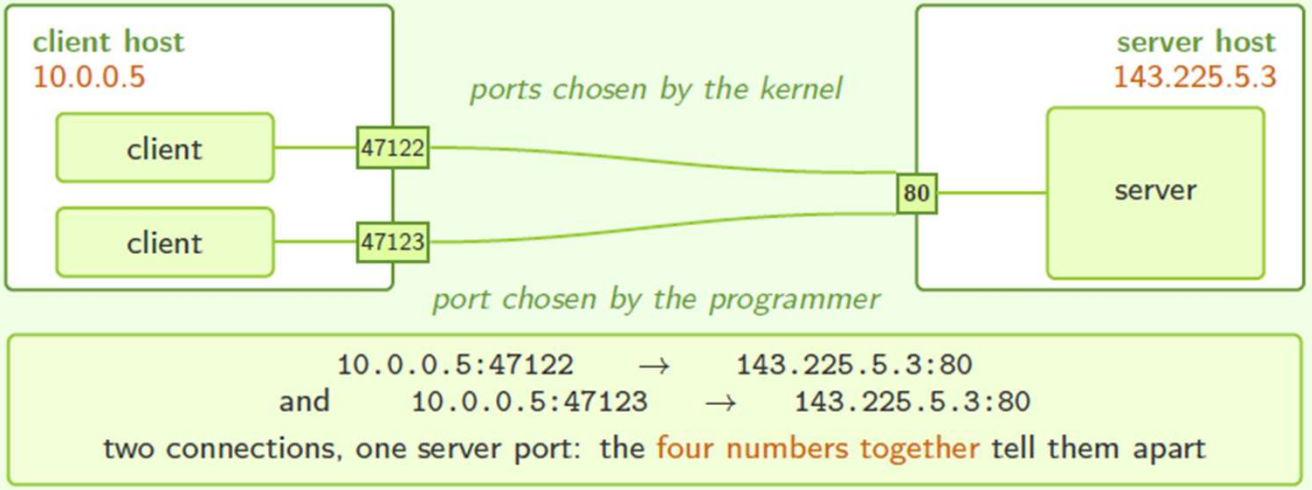
- The accept() call returns a new connected socket and fills cliaddr with the remote client's IP address and port number.

- Subsequent TCP segments are demultiplexed to the appropriate connected socket using the source IP address, source port, destination IP address, and destination port.



Transport Layer

Multiplexing/Demultiplexing in TCP



accept creates the second socket, and the transport layer records the four values it read in the arriving segment. Every later segment whose four values match is handed to that socket — which is why the listening one can keep listening.

46428

Transport Layer

Multiplexing/Demultiplexing in TCP

- An established TCP connection is identified by a 4-tuple:
  - Source address (IP).
  - Source port.
  - Destination address (IP).
  - Destination port.

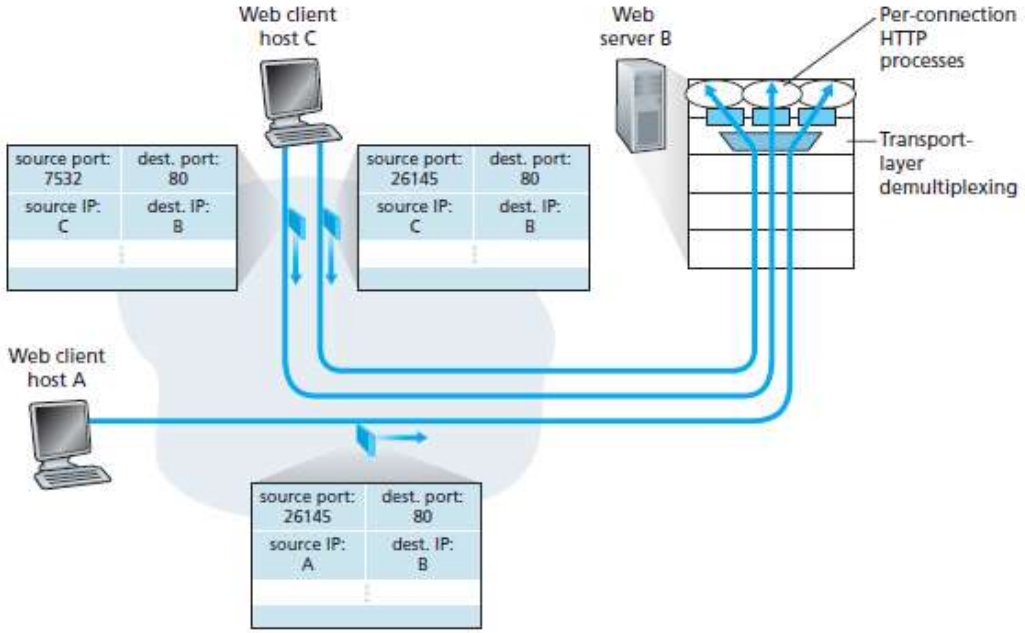
- Each connected TCP socket is associated with a single TCP connection between two endpoints.
- Multiple connections can share the same server port, but they are distinguished by their 4-tuples.

Transport Layer

Multiplexing/Demultiplexing HTTP example

- We can then have multiple processes from one machine communicating with multiple processes (or with the same process) on another.
- In this example **a host C initiates two HTTP sessions to server B**, and **host A initiates one HTTP session to B**. Hosts A and C and server B each have their own unique IP address (A, C, and B). Host C assigns two different source port numbers (26145 and 7532) to its two HTTP connections.

- Host **A may assign a source port of 26145** to its HTTP connection (not aware of C).
- This is not a problem, since **the two connections have different source IP** addresses.
- In this example, the server opens a new process (or a new thread) for each incoming connection by assigning to it the specific socket (new_socket).
- This is quite typical but opening too many processes (or threads) may impair the performance of the server.



Transport Layer
UDP

- As already explained, **UDP is useful for simple and rapid connections**. It mainly performs 2 tasks:
  - Process-to-process delivery (ports, multiplexing and demultiplexing).
  - Error Checking.

- UDP is connectionless: **there is no handshaking** between sending and receiving transport-layer entities before sending a segment.

- UDP has **no guarantee about delivery of messages** (no reliable data transfer implemented) and has **no congestion/flow control**.

- Roughly speaking, UDP is a **minimalistic protocol** that just adds “the essential” with respect to the lower-level IP protocol.

Transport Layer
UDP

- Why to use UDP instead of TCP?
  - **Application-level control**: UDP is more direct, so the application has more control on the transmission.
    - This can be useful for instance in real-time applications, where sending rate or delays are crucial while data loss can be tolerated, so less checking is preferred.
  - **Fast connection establishment**: there is no handshake, so less delay to establish a connection.
    - This is good for instance in DNS to not provide additional delays.
  - **No connection state**: there are no congestion-control parameters, no sequence nor acknowledgment numbers.
    - A server can typically support many more active clients when the application runs over UDP rather than TCP.
  - **Minimal packet overhead**: The TCP segment adds 20 bytes of header in every segment, whereas UDP only adds 8 bytes.

Transport Layer

UDP: DNS Example

- An example of application using UDP connection is DNS. The DNS client works as follows:
  - In **application layer**: when the host wants to make a query to a DNS server, it constructs a DNS query message and passes the message to UDP.
  - In **transport layer**: without performing any handshaking with the DNS server, UDP adds header fields to the message and passes the resulting segment to the network layer.
  - In **network layer**: the UDP segment is encapsulated into an IP datagram and sent to the DNS server (through the access layer).
  - The client then waits for a reply to its query. If **reply is not received** (possibly because the underlying network lost the query or the reply), it might try different approaches:
    - Resending the query.
    - Sending the query to another name server.
    - Informing the invoking application that a reply is not received.

Transport Layer

UDP: Usage

- Applications needing **reliable data transfer** such as, e-mail, remote terminal access, the Web, run over TCP (or over reliable UDP-based protocols like QUIC).
- In this cases **we can't afford packet loss**, for instance, mails must be fully delivered we can't have missing elements.

- **SNMP** uses UDP to carry out device management. Network management applications **must often run when the network is in a stressed state** (precisely when reliable data transfer is difficult).
- **Multimedia applications** such as Internet phone, video conferencing, streaming, often use both UDP and TCP, because small amount of packet loss can be tolerated.
- In general, real-time applications react very poorly to TCP congestion control.

Service	Application Protocol	Transport Protocol
E-mails	SMTP	TCP
Remote terminal	Telnet	TCP
Web	HTTP	TCP or UDP
File transfer	FTP	TCP
Name translation	DNS	UDP
Network device management	SNMP	UDP
Multimedia streaming	Proprietary	UDP or TCP
Internet telephony	Proprietary	UDP or TCP

Transport Layer

UDP: Usage

- On the other hand, running **multimedia applications over UDP is controversial** because no congestion control is performed.

- If everyone were to (selfishly) start streaming high-bit-rate video without using any congestion control, **network devices would overflow** causing more UDP packets to be lost.

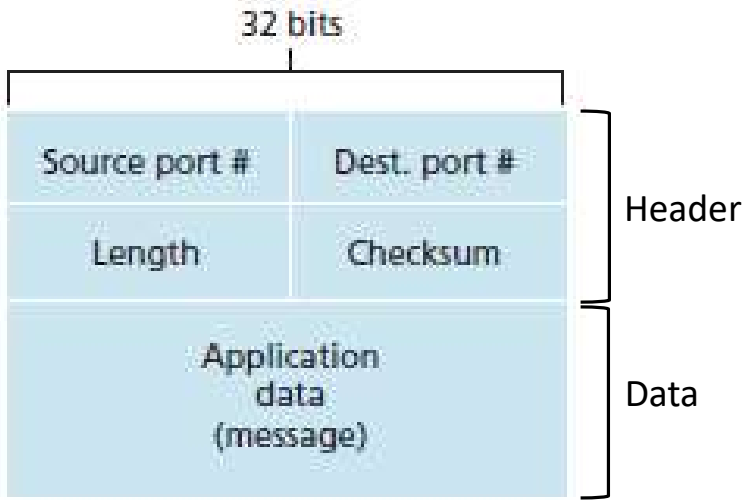
- High loss rates induced by the uncontrolled UDP senders would also cause the **TCP senders to dramatically decrease their rates**.

Service	Application Protocol	Transport Protocol
E-mails	SMTP	TCP
Remote terminal	Telnet	TCP
Web	HTTP	TCP
File transfer	FTP	TCP
Name translation	DNS	UDP
Network device management	SNMP	UDP
Multimedia streaming	Proprietary	UDP or TCP
Internet telephony	Proprietary	UDP or TCP

Transport Layer

UDP: Segment Format

- The **UDP segment** (or datagram to be more precise) is composed by 5 elements:
  - **UDP header** (8 bytes) which includes information related to the UDP protocol and is composed by **4 elements of 16 bits** each:
    - **Source port**: port number of the sending process.
    - **Destination port**: port number of the receiving process.
    - **Length**: length of the whole datagram in bytes (header+data, i.e., N + 8 bytes).
    - **Checksum**: used by the receiver to check if the message is intact.
  - **Data** (N bits) which is the actual message from the application layer.

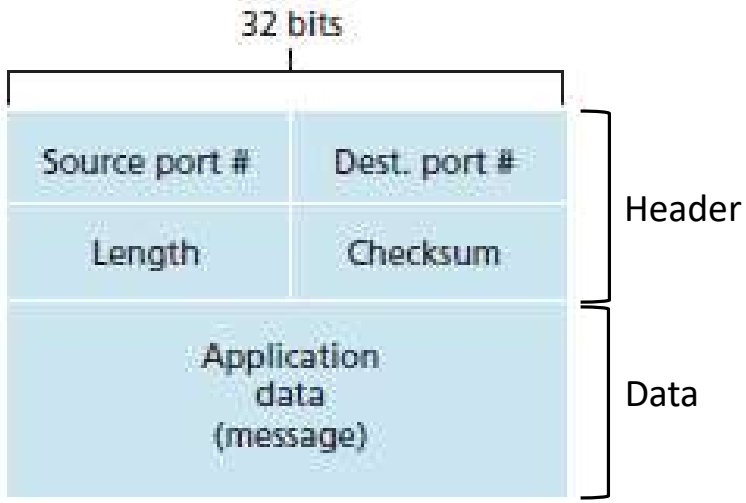


Transport Layer

UDP: Segment Format

- The UDP header contains a 16-bit Length field, specifying the total size of the UDP datagram
  - $\text{UDP Length} = \text{UDP Header} + \text{UDP Payload}$
  - A 16-bit field can represent values from 0 to $2^{16} - 1$ = 65535
  - Since the UDP header occupies 8 bytes, the maximum theoretical UDP payload is: (65535-8=65527) bytes

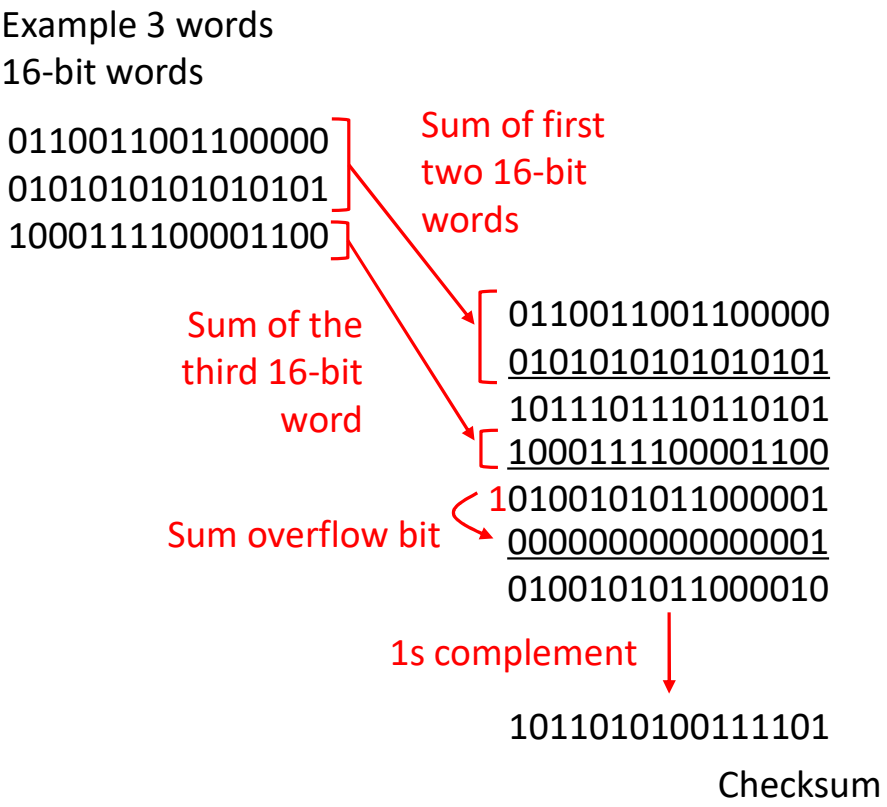
Note: The IP layer imposes additional constraints on the maximum UDP datagram size. In IPv4, the maximum UDP payload is 65,507 bytes (assuming a 20-byte IP header).



Transport Layer

UDP: Checksum

- Integrity checking is performed by using the **checksum**. The UDP checksum is a 16 bits filed used for error detection as follows:
  - UDP at the sender side performs the **1s complement of the sum of all the 16-bit words** in the datagram (+ IP pseudo-header), with **any overflow bit being summed** itself (end-around carry).
  - This **result is put in the checksum field** of the UDP datagram.
  - When **the receiver sums all bits inside the message (checksum included)** the sum must be 1111111111111111 (16 ones).
  - If one bit is 0, then we know that **errors have been introduced** into the packet.



Transport Layer

UDP: Checksum

- Integrity checking is performed by using the **checksum**.
- An N-bit checksum detects a single-bit error, but does not guarantee detection of two-bit (or more) errors.
- Two-bit errors: two bit flips may produce opposite changes that cancel each other, leaving the checksum unchanged
- Example:

	Transmitted	Received
Word 1	0001	0000
Word 2	0000	0001
	----	----
Sum	0001	0001
Checksum	1110	1110

Transport Layer

UDP: Checksum

UDP Checksum: Why End-to-End Error Detection?

- **Link-layer error detection is not sufficient**: not all network links necessarily provide the same integrity checks.
- **Errors may occur inside intermediate devices**: data can be corrupted inside a router, even when the incoming and outgoing link-layer checks succeed.
- **End-to-end principle**: integrity checks that must hold between communicating endpoints cannot rely exclusively on intermediate network components.
- **Complementary mechanisms**: link-layer and transport-layer error detection protect data at different points along the communication path.

UDP detects errors but does not correct them: a datagram with an invalid UDP checksum is silently discarded. UDP provides no retransmission mechanism.

Transport Layer

UDP: Checksum - Limitations

Can the UDP checksum detect all transmission errors?

No. Different data contents may produce the same checksum.

- **Reordering of 16-bit words:** swapping two words does not change their sum, so the error goes undetected.
- **Compensating errors:** if one word increases by a certain amount and another decreases by the same amount, the checksum may remain unchanged.
- **Limited error detection:** the checksum is computationally inexpensive and detects many common errors, but it does not guarantee data integrity.

Stronger error detection mechanisms, such as the Cyclic Redundancy Check (CRC), are commonly used at the data link layer.

Note: Link-layer checks complement, but do not replace, end-to-end error detection.

Transport Layer

UDP: Delivery, Loss and Ordering

UDP provides a **connectionless, best-effort delivery service**.

- **No delivery guarantee:** UDP datagrams may be lost during transmission.
- **No retransmissions:** UDP does not automatically retransmit lost or corrupted datagrams.
- **No ordering guarantee:** datagrams may arrive in a different order from the one in which they were sent.
- **No duplicate suppression:** UDP does not guarantee that a datagram will be delivered only once.

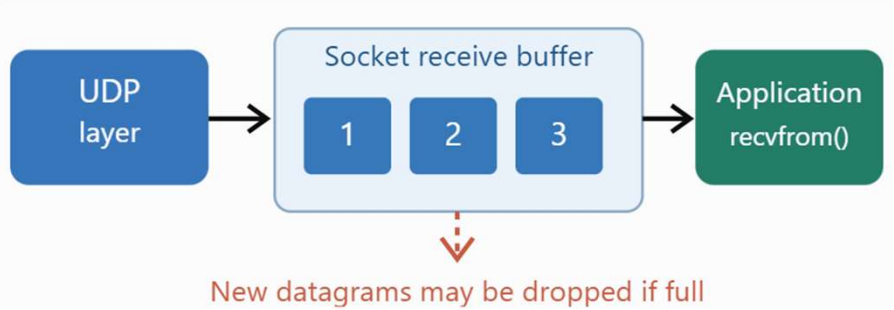
Applications requiring reliability must implement additional mechanisms or use a transport protocol such as TCP.

Transport Layer

UDP: Socket Buffering and Datagram Loss

UDP: Socket Buffering and Datagram Loss

- Incoming UDP datagrams are **demultiplexed to the appropriate socket** and placed in its receive buffer.
- The application retrieves datagrams by calling recvfrom().
- If datagrams arrive faster than the application can process them, the **receive buffer may fill up**.
- When the buffer is full, additional incoming datagrams may be **discarded**.



UDP provides no flow control: the sender is not automatically slowed down when the receiver cannot keep up.

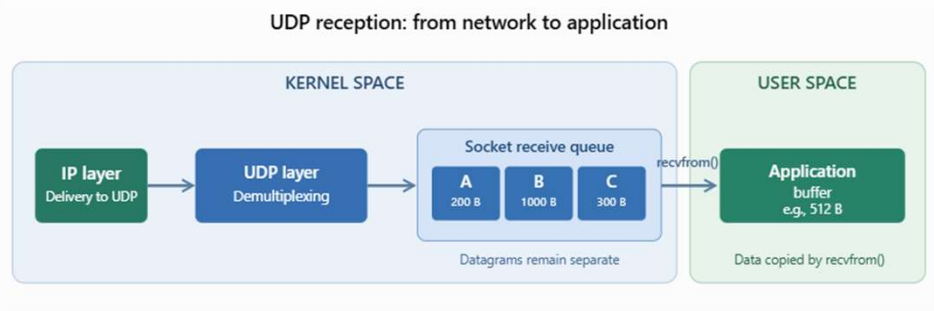
Note: A successful sendto() call does not guarantee that the datagram has reached the destination application.

Transport Layer

UDP: Socket Buffering and Datagram Loss

UDP Reception: Buffering and Datagram Loss

- The UDP layer demultiplexes incoming datagrams and queues them in the appropriate socket receive buffer (kernel space).
- The application retrieves one datagram at a time using recvfrom().



Two possible problems:

- Socket receive buffer full: an incoming datagram may be dropped entirely.
- Application buffer too small: the datagram is truncated, and the remaining bytes are discarded.

- **UDP preserves message boundaries:**

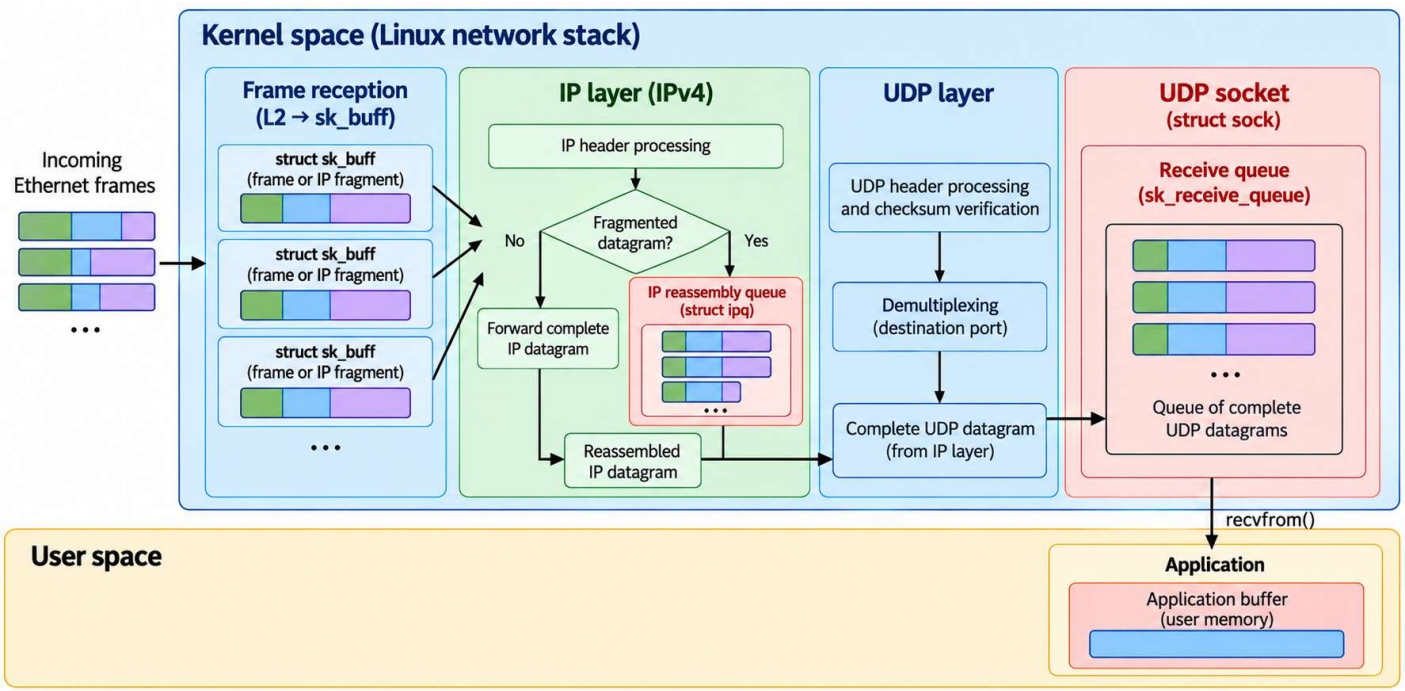
consecutive datagrams are never merged into a single byte stream.

Transport Layer

UDP: Reception Path in Linux

UDP Reception Path in Linux

- A packet is received by the network interface and stored in **kernel memory**.
- The **IP layer** processes the packet and performs **reassembly** if fragmentation occurred.
- The **UDP layer** checks the UDP header and checksum, then **demultiplexes** the datagram to the appropriate socket.
- The datagram is placed in the **socket receive queue**.
- The application reads the datagram through **recvfrom()**, which copies the data into the **application buffer**.



Transport Layer

UDP: Application-Level Reliability

UDP does not provide acknowledgments, retransmissions, or in-order delivery.

Applications can implement these mechanisms when needed:

- **Sequence numbers:** identify messages and detect missing or duplicated data.
- **Acknowledgments:** confirm successful reception.
- **Timeouts and retransmissions:** recover from missing responses.
- **Congestion control:** regulate the amount of traffic injected into the network.

Examples:

- DNS clients may retry unanswered queries.
- QUIC implements reliability and congestion control on top of UDP and supports HTTP/3.

UDP provides a minimal transport service, leaving additional functionality to higher-layer protocols.

Transport Layer

UDP: summary

- The transport layer runs only on the end systems: it connects processes, while the network layer connects hosts.
- Demultiplexing is choosing the socket by reading the header; multiplexing is the same job on the way out.
- A UDP socket is identified by two numbers, a TCP socket by four — which is why one accepts and the other does not.
- UDP adds to IP only ports and a checksum, and is chosen for control, speed of setup, absence of state and a small header.
- The checksum detects and does not repair, and misses errors that cancel or reorder.